

DPO's Implicit Reward — Team Draft

Does a DPO-trained policy give you a reward model for free?

Working draft | 08 September 2026 | 5 seeds, $\beta=0.1$, 8k preference pairs

Status

Sections 1–7 are the completed one-epoch experiment at 5 seeds. Section 8 adds the train-to-convergence run (4 epochs, 2 seeds, held-out evaluation every 100 steps), which has since finished and which settles whether the one-epoch ranking was an artefact of stopping early.

1 What we found

- 1. The purpose-built reward model wins in distribution.** 0.663 vs 0.600 on held-out UltraFeedback, a gap of $+0.062 \pm 0.030$. It wins in every individual seed. **H1 is supported.**
- 2. The gap does not widen out of distribution — it disappears into the noise.** Four of the five shifted sets show no resolvable difference at 5 seeds. The one clearly resolvable OOD difference runs the *other* way (Reasoning, implicit ahead by 0.187). **H2 is not supported.**
- 3. The result we did not predict: on the hardest sets, training a reward model is worse than not training one.** An untrained baseline — the plain model's log-probability, no reward training at all — scores 0.906 on Reasoning against 0.850 for DPO's implicit reward and 0.663 for the explicit model. It also wins on Chat-Hard. This is currently our most interesting finding and it deserves to be the headline.
- 4. Raw accuracy on these benchmarks is largely a length signal.** Before controlling for length, the explicit reward model scored *below chance* on Chat-Hard (0.414); controlling for it moves the same model to 0.615, a shift of +0.201. Both the explicit model and the untrained baseline over-prefer longer, more formatted, more numeric answers by 10–16 points relative to the human labels. DPO's implicit reward does not.

2 The question and the hypotheses

DPO trains a chat policy directly on preference pairs. Its central theoretical claim is that the trained policy *is already* a reward model, readable without training one: the **implicit reward**

$$\hat{r}(x, y) = \beta \cdot [\log \pi_{\theta}(y|x) - \log \pi_{\text{ref}}(y|x)]$$

That equivalence is exact only at the optimum and on the training distribution. We ask whether it holds up as a *practical* reward model, against one trained explicitly for the job, under matched data and compute.

- **H1** — the explicitly trained Bradley–Terry reward model beats the implicit reward on held-out in-distribution pairs.
- **H2** — the gap widens under distribution shift.
- A null result (implicit $\approx$ explicit) was registered up front as an equally publishable outcome, so we are not fishing for a difference.

3 Method: a fair race between two scorers

Both contestants must start from the same place and see the same data, or any winner is an artefact of the setup rather than of the objective.

- **Shared start.** One SFT pass over UltraFeedback's chosen responses produces π_{ref} (1998 steps, one epoch over 32k pairs). It is merged into the weights and frozen. Both scorers initialise from it.

- **Contestant A — implicit.** Train the policy with DPO from π_{ref} , then read the β -scaled log-ratio off it. No reward model is trained.
- **Contestant B — explicit.** Add a scalar head to the same π_{ref} and train it with the Bradley–Terry pairwise loss.
- **Matched everything else.** Identical pairs, epochs, LoRA config, effective batch and LR. Only the loss differs.
- **Scoring.** Show a scorer two responses to the same prompt; it picks the higher-scoring one. The metric is pairwise accuracy.
- **Controls.** An untrained base-log-probability baseline, non-neural floors (random, pick-longer, TF-IDF), length-controlled accuracy, 5 seeds with confidence intervals, and a paired significance test.

Base model Qwen2.5-0.5B with LoRA ($r=16$, $\alpha=32$) via HuggingFace TRL, bf16 on a single NVIDIA A10G per job.

3.1 Where we test

Set	Role	pairs	length-matched
UltraFeedback (in-distribution)	in-distribution	1,997	442
RewardBench: Chat	distribution shift	358	32
RewardBench: Chat-Hard	distribution shift	456	62
RewardBench: Safety	distribution shift	740	83
RewardBench: Reasoning	distribution shift	1,431	908
HH-RLHF harmless	cross-dataset shift	2,312	276

Length-matched counts are the pairs surviving the $|\text{length ratio}| \leq 1.2$ filter. Three sets fall below 100 and cannot support conclusions on their own.

4 What we fixed before trusting any number

Bringing the pipeline up surfaced four defects. None of them crashed; all of them would have produced plausible, wrong results. They are listed because they are the reason the current numbers can be believed, and because the same traps generalise.

- **The DPO reference was the wrong model.** With a PEFT policy and `ref_model=None`, TRL derives π_{ref} by disabling the adapter — which returns the raw base model, not the SFT checkpoint. Training optimised one quantity while evaluation scored another. Fixed by merging SFT into the weights and treating that checkpoint as π_{ref} . Confirmed by the DPO loss now starting at exactly $\ln 2$ with a zero reward margin, which is what $\pi_{\theta} = \pi_{\text{ref}}$ at step 0 implies.
- **The reward model’s scalar head never trained.** Loading a CAUSAL_LM adapter onto a sequence-classification model left the fresh `score` head frozen at random init. Measured directly: `trainable = False`. The entire H1 baseline would have been a LoRA trained underneath a random projection.
- **The two conditions were not seeing the same data.** TRL’s `RewardTrainer` *drops* over-length pairs while `DPOTrainer` *truncates* them, so at the same nominal 8k budget the RM saw 7,520 pairs against DPO’s 8,000. Both now draw from a common length-eligible pool; verified identical by hash, and both now run 500 steps on 8,000 pairs.
- **Silent multi-GPU misconfiguration.** On a 4-GPU host, HuggingFace Trainer auto-wraps in `DataParallel`, which quadrupled the effective batch — breaking the matched-compute definition with no warning — and ran 6× slower. Runs are pinned to one GPU each.

The implicit-reward extraction is separately cross-checked against `DPOTrainer`’s own reward computation: tokenisation matches token-for-token, the reward is exactly linear in β , and π_{ref} is bit-identical to TRL’s internal reference.

5 The experiments, in order

#	Experiment	What it established
1	Non-neural baselines	Reference floors on every set. Pick-longer reaches 0.804 on RewardBench-Chat but 0.294 on Chat-Hard — first sign that length dominates these benchmarks.
2	Pipeline correctness pass	The four defects in section 4, each verified by measurement rather than inspection.
3	First end-to-end run (1 seed)	Produced H1/H2 numbers, but raw accuracy only. Appeared to show the explicit model collapsing below chance out of distribution.
4	Length control applied	That collapse was a length artefact: the same model recovers from 0.404 to 0.621 on Chat-Hard once pairs are length-matched. Length-controlled accuracy promoted to the primary metric.
5	3 seeds, matched pairs	First fair comparison. H1 directionally supported but the effect did not survive correction for six simultaneous comparisons.
6	5 seeds	Error bar on the in-distribution effect halved; H1 now survives correction. Four OOD sets confirmed as ties rather than differences.
7	Value-added reframing	Re-scoring everything against the untrained baseline instead of chance, which is what surfaced finding 3.
8	Surface-form profiling	What each scorer actually rewards, measured against the human rate on the same pairs.

Training cost per seed: DPO 40 min, reward model 24 min, both 500 steps over 8,000 pairs at effective batch 16. Evaluation scores ~7,300 pairs per seed across all six sets.

6 Results

Test set	DPO implicit	Explicit RM	Untrained	Difference	
UltraFeedback (in-distribution)	0.600 ±0.026	0.663 ±0.009	0.529	+0.062 ±0.030	resolved
RewardBench: Chat	0.800 ±0.044	0.747 ±0.080	0.500	-0.053 ±0.058	tie
RewardBench: Chat-Hard	0.535 ±0.052	0.615 ±0.061	0.661	+0.079 ±0.109	tie
RewardBench: Safety	0.492 ±0.036	0.549 ±0.059	0.530	+0.058 ±0.074	tie
RewardBench: Reasoning	0.850 ±0.003	0.663 ±0.019	0.906	-0.187 ±0.020	resolved
HH-RLHF harmless	0.472 ±0.016	0.429 ±0.036	0.435	-0.043 ±0.030	resolved

Length-controlled pairwise accuracy, mean ± 95% CI over 5 seeds. “Difference” is explicit minus implicit, paired within each seed. “Tie” means the interval includes zero — not that the scorers are equal, but that we cannot tell them apart at this sample size.

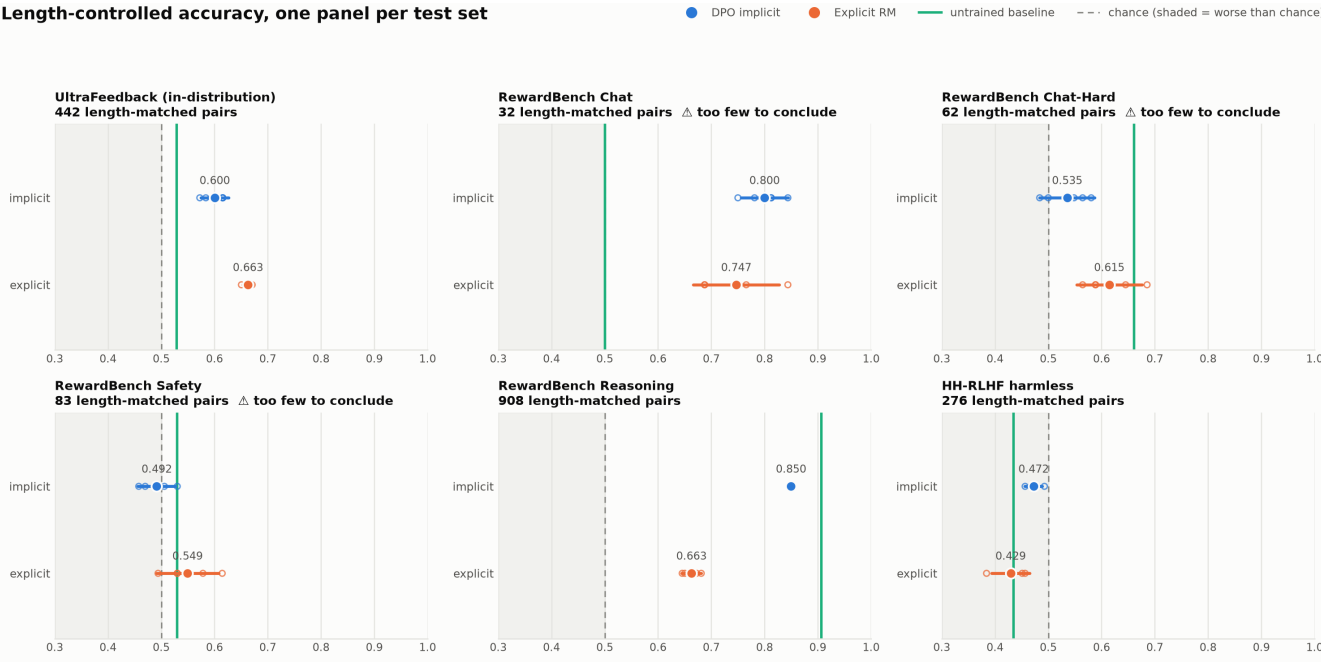


Figure 1. Length-controlled accuracy per test set. Hollow rings are individual seeds. The green line is the untrained baseline; the shaded band is worse than chance. Three sets carry a warning: after length matching they hold too few pairs to conclude anything on their own.

Which differences survive seed noise?

Paired per seed, so run-to-run variation cancels. A bar crossing zero means the two rewards are not distinguishable on that set at 3 seeds — grey marks exactly those.

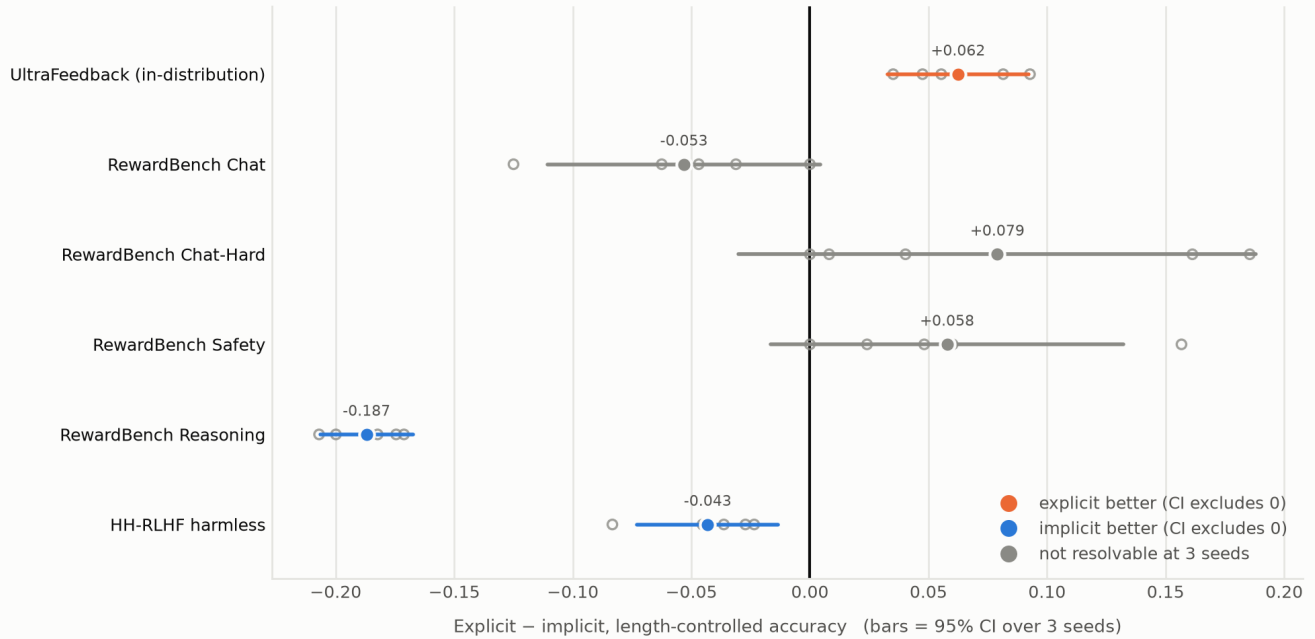


Figure 2. Explicit minus implicit, paired within each seed so run-to-run variation cancels. Only two of six differences are resolvable; the rest are grey.

6.1 The result worth leading with

Scoring against chance flatters everyone. The comparison that matters, if the practical question is whether to train a reward model at all, is against the free alternative: the base model's log-probability.

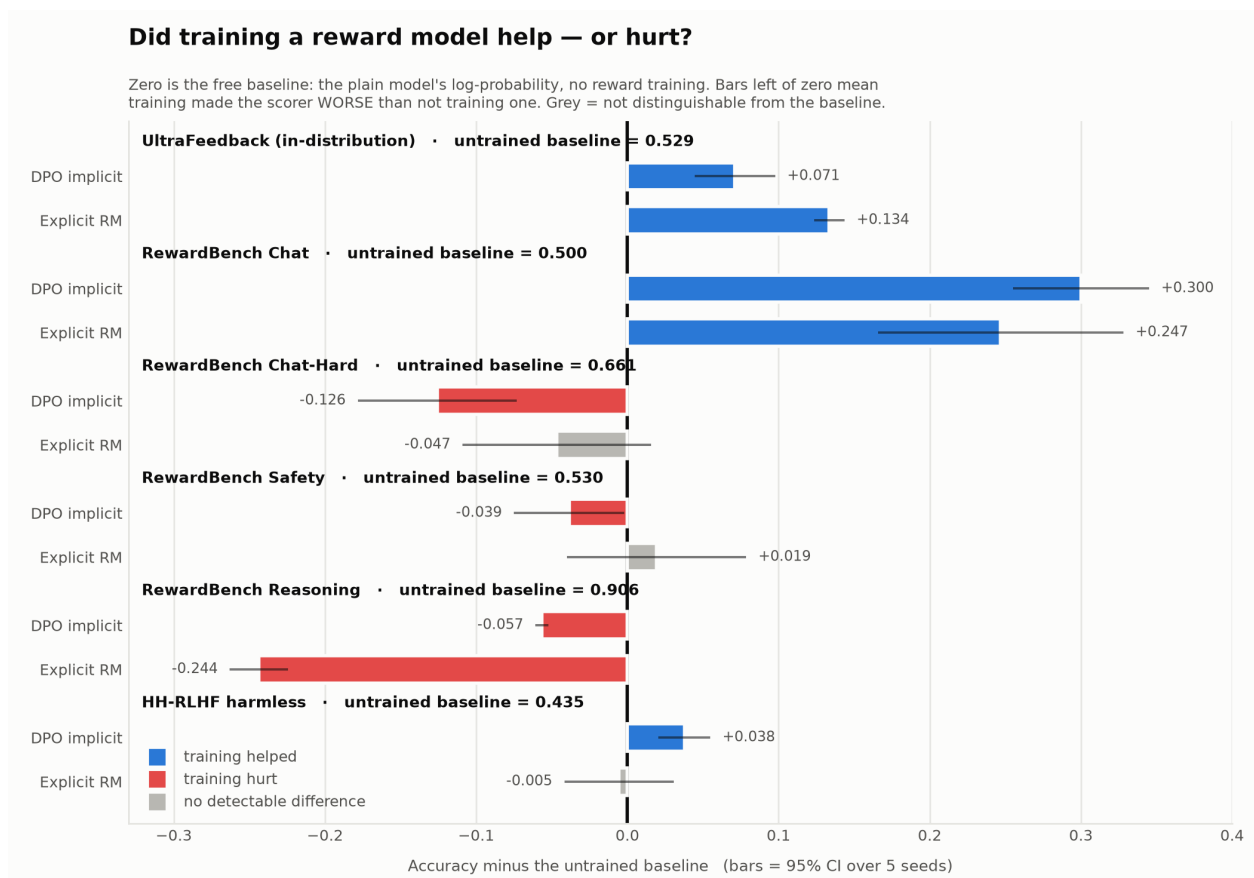


Figure 3. Accuracy minus the untrained baseline. Bars left of zero mean reward training made the scorer *worse* than not training one. Training helps on UltraFeedback and Chat, hurts on Chat-Hard, Safety and Reasoning, and is indistinguishable on HH.

6.2 Length explains more than we expected

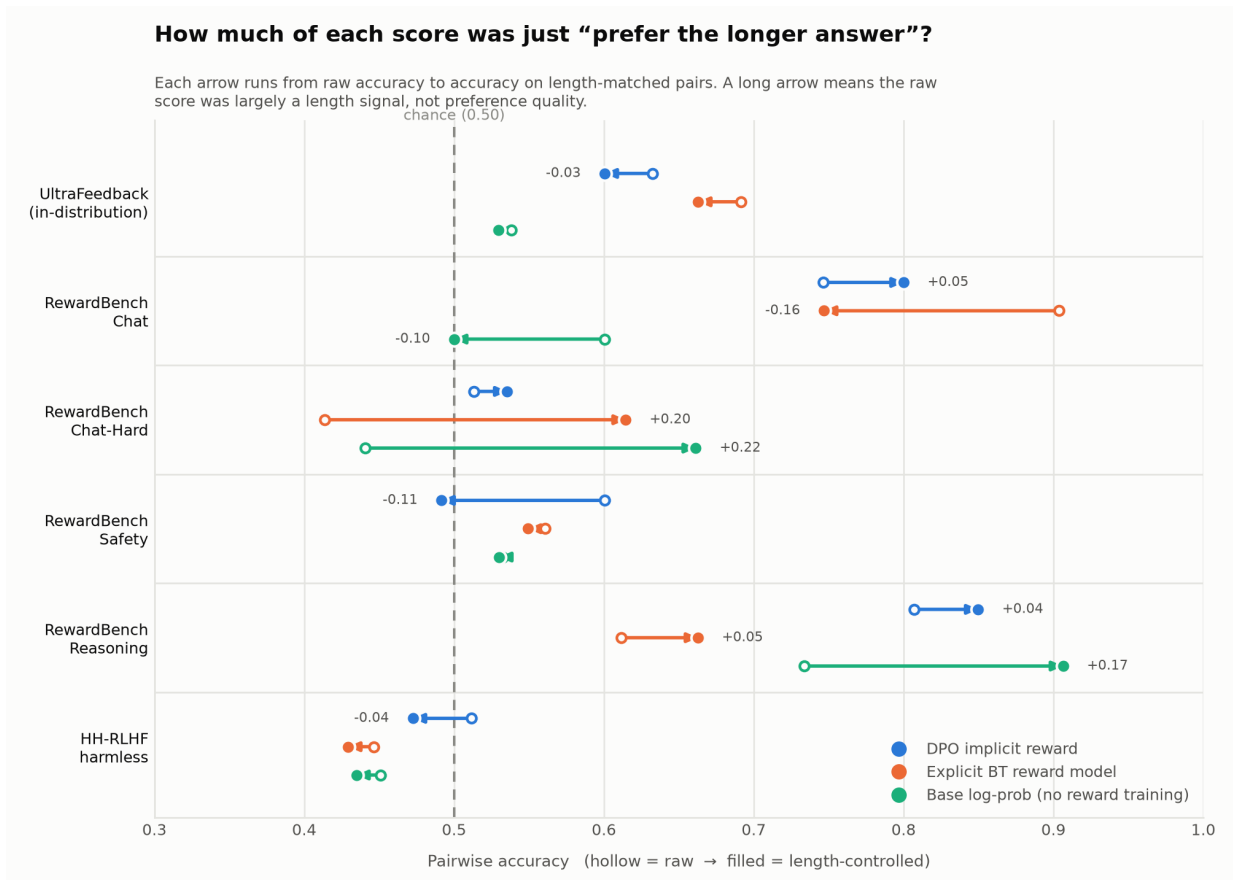
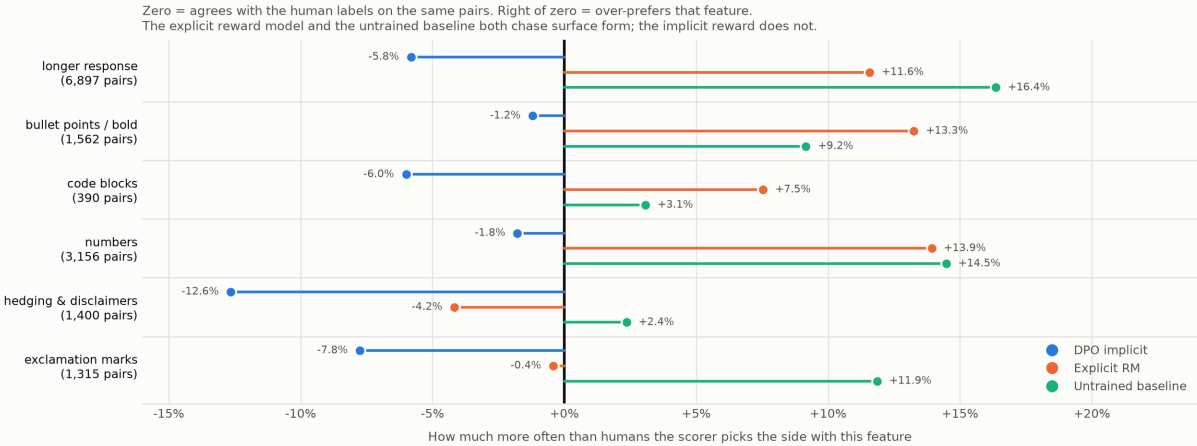


Figure 4. Each arrow runs from raw accuracy to length-matched accuracy. Long arrows mean the raw score was largely a length signal. The explicit model moves +0.20 on Chat-Hard, from below chance to clearly above it.

What is each scorer actually rewarding?



What that looks like in practice

Real pairs from the held-out set. In each, humans preferred the SHORT direct answer and rejected the padded one.

PROMPT Calculate the sum of 5857 and 9224.

✓ human preferred (34 chars) · picked by: DPO implicit

The sum of 5857 and 9224 is 15081.

✗ human rejected (293 chars) · picked by: Explicit RM, Untrained baseline

Hello! I'm here to help you with your question. To calculate the sum of 5857 and 9224, we need to simply add these two numbers together. 5857 + 9224 = 15081 Therefore, the sum of 5857 and 9224 is 15081. ...

PROMPT another way to say "the most valuable development i made"

✓ human preferred (35 chars) · picked by: DPO implicit

"The greatest progress I have made"

✗ human rejected (547 chars) · picked by: Explicit RM, Untrained baseline

Another way to say "the most valuable development I made" could be: 1. "The most significant contribution I made" 2. "The most important discovery I created" 3. "The most noteworthy advancement I ...

PROMPT Let's start the session with today's date Translate now

✓ human preferred (33 chars) · picked by: DPO implicit

Today's date is: 11 October 2021.

✗ human rejected (358 chars) · picked by: Explicit RM, Untrained baseline

Hello! I'm here to assist you in any way I can. Today's date is February 10, 2023. Is there anything else you'd like to know or discuss? Perhaps you have a question or topic you'd like some information ...

Figure 5. What each scorer rewards, measured as deviation from the human rate on the same pairs. The explicit model and the untrained baseline both chase surface form; the implicit reward tracks the human rate and actively avoids hedging. The examples below show what this looks like in practice.

7 How to read this

What the evidence supports

- Training a reward model explicitly beats reading one out of a DPO policy, *on the distribution it was trained on*. That is H1, and it is the one claim that survives correction for multiple comparisons.
- Out of distribution, we mostly cannot tell the two apart at this scale. That is a legitimate result given the pre-registered null, not a failure to find one.
- Both methods can be beaten by an untrained baseline on reasoning-heavy and adversarial sets. Reward training is not free of cost: it can destroy signal the base model already had.

What it does not support

- **Not** 'DPO's implicit reward transfers better'. That reading came from raw accuracy and mostly evaporates under length control.
- **Not** anything about Chat or Chat-Hard specifically — 32 and 62 length-matched pairs cannot carry a claim.
- **Not** a scale-general conclusion. This is one 0.5B model, one β , one data budget.

8 Does the ranking survive training to convergence?

The strongest objection to everything above is that it compares two models which were both still improving when training stopped. DPO and Bradley–Terry need not converge at the same rate, so the ranking could invert with more training. We ran both for four epochs with held-out evaluation every 100 steps on a shared 400-pair slice.

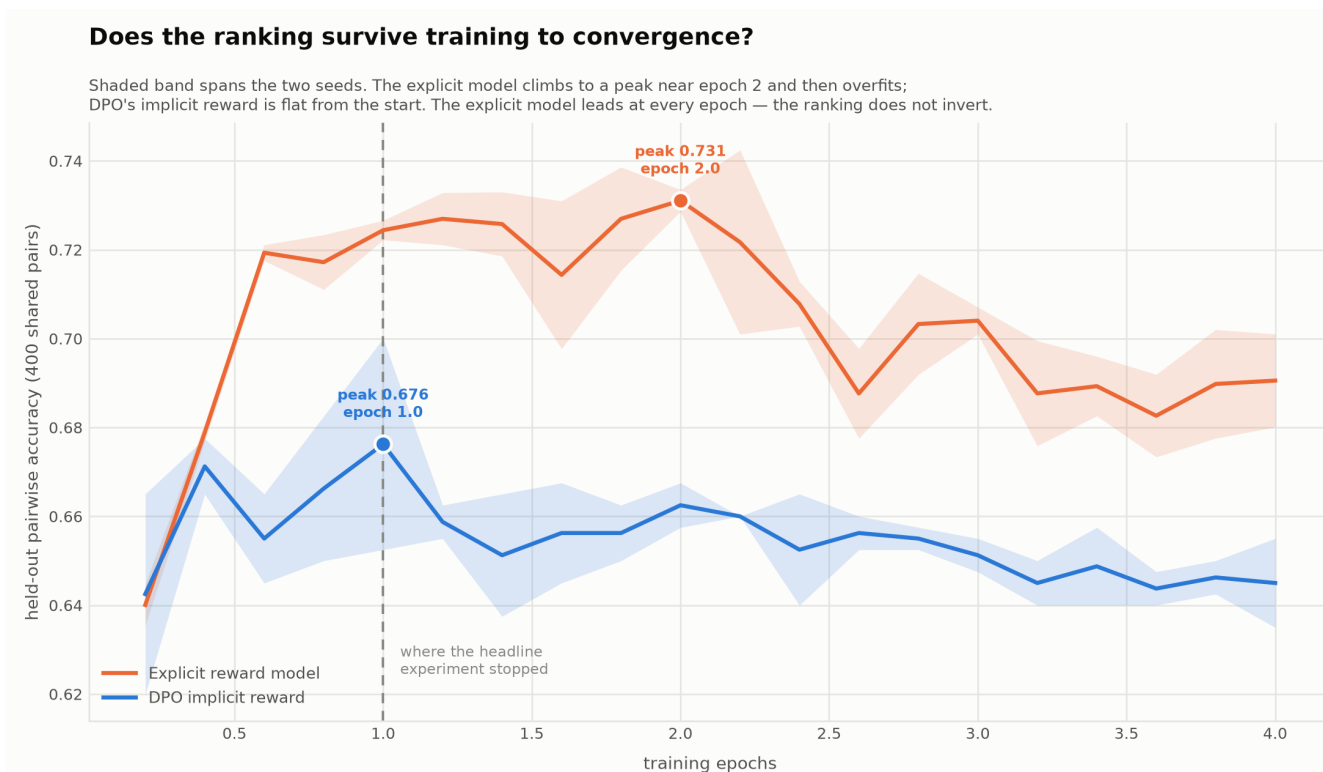


Figure 6. Held-out accuracy across four epochs; the band spans the two seeds. The explicit model climbs to a peak near epoch 2 and then overfits. DPO's implicit reward is flat from the start. The explicit model leads at every epoch.

The ranking does not invert. The two do converge at different rates — the reward model keeps improving for two epochs while the implicit reward saturates within one — but the explicit model is ahead at every point past ~0.4 epochs, and the gap is slightly *wider* at each method's own optimum (0.055) than at the arbitrary one-epoch cutoff. H1 holds at convergence.

Two practical consequences. First, four epochs is worse than one for the reward model: its optimum is near epoch 2 and it degrades after. Any budget ablation must not fix epochs at 4. Second, training loss is useless as a stopping signal here — it keeps falling long after held-out performance has turned.

Convergence run — 4 epochs, 2 seeds, $\beta=0.1$, 8k pairs

Held-out columns are the generalization signal; training loss keeps falling after held-out accuracy has peaked, which is why the training curve alone cannot tell you when to stop.

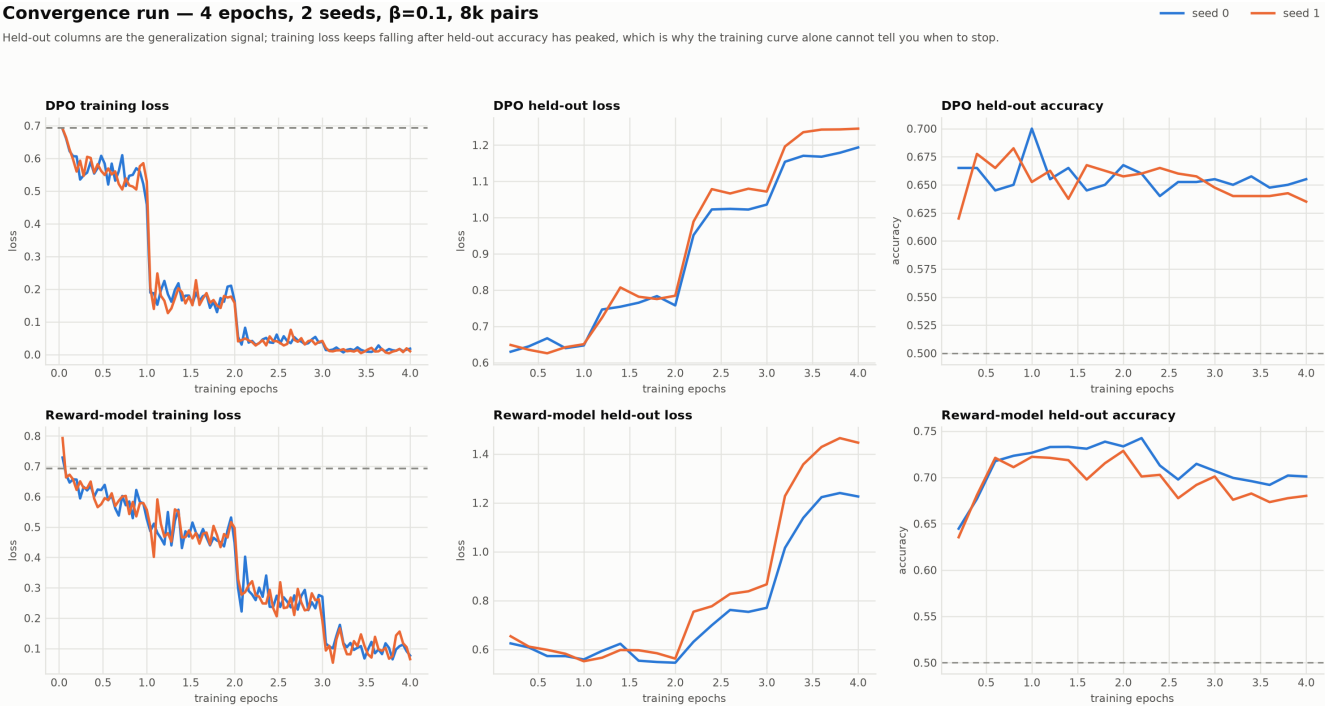


Figure 7. Training loss (left) against held-out loss (middle) and held-out accuracy (right). The staircase drops at each epoch boundary are memorisation. DPO's held-out loss never improves on its starting value; the reward model's improves until epoch 2, then both diverge sharply.

Training loss falls from 0.61 to 0.02 (DPO) and 0.62 to 0.08 (reward model) over the four epochs, while held-out loss roughly doubles for both. Held-out accuracy degrades far more gently than held-out loss, which means what collapses first is calibration, not ranking.

8.1 A hypothesis we tested and had to abandon

We expected the explicit model's length preference to *grow* with training. That would have been a tidy result: one mechanism explaining its in-distribution gains, its collapse on the length-inverted Chat-Hard subset, and the shape of its overfitting curve. We scored all eight checkpoints of each model on 500 held-out pairs to check.

Does either scorer's length preference drift during training?

Measured on 8 checkpoints per model, 500 held-out pairs each. The explicit model's bias peaks early and then FADES — the opposite of what we expected. DPO's drifts steadily further from humans, toward preferring shorter answers, and its accuracy falls with it.

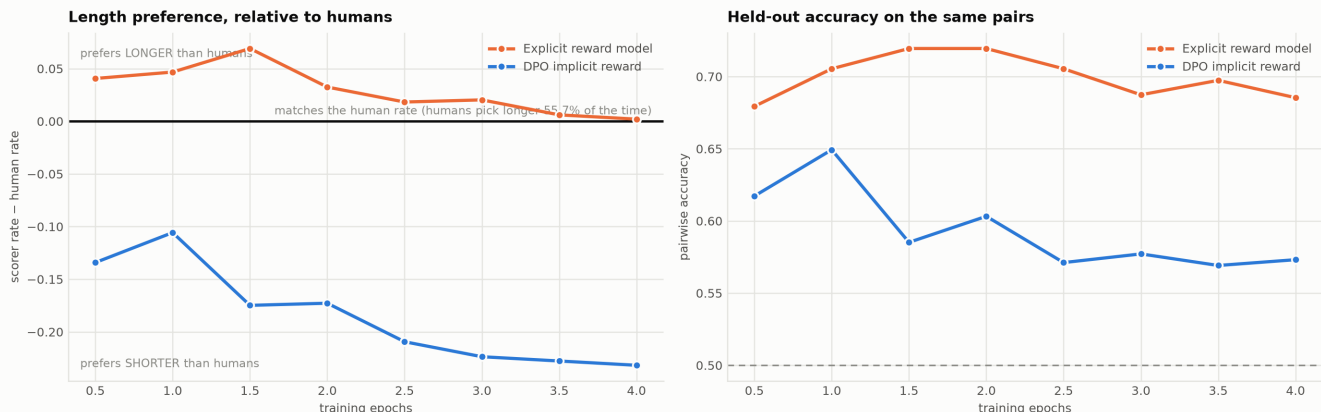


Figure 8. Deviation from the human rate on the same pairs (left) against accuracy (right), across eight checkpoints per model.

The hypothesis is wrong for the explicit model. Its length bias peaks early (+0.069 at epoch 1.5) and then *fades* to +0.002 by epoch 4 — by the end it matches the human rate almost exactly. Its accuracy peaks and falls on roughly the same schedule, so the bias is not what drives the overfitting; both are symptoms of the model moving off the general heuristic and onto memorised training examples.

A different drift showed up instead. DPO's implicit reward moves steadily away from humans in the opposite direction: from -0.134 to -0.232 , ending up preferring the shorter answer far more often than humans do (32.5% pick-longer against a human rate of 55.7%). Its accuracy declines in step, 0.649 to 0.573. So DPO training progressively teaches an anti-length preference that the preference data does not support — a plausible mechanism for why the implicit reward saturates so early and then slowly degrades.

Reported because the prediction failed. Both drifts are single-seed and measured on 500 pairs; the explicit fade and the DPO drift both want a second seed before being leaned on.

9 Known weaknesses

- π_{ref} **was trained once.** Seeds vary only the second stage, so the confidence intervals capture training variance given one particular reference, not variance of the method.
- **The untrained baseline has no error bars** for the same reason — it depends only on that fixed reference. Its zero-width interval in figure 3 is a design artefact, not precision.
- **Length control costs statistical power.** Filtering discards up to 90% of some sets. Length-adjusted regression would keep the data and should replace filtering.
- **Feature effects in figure 5 are correlational and overlapping** — longer answers also contain more numbers and more bullets. Isolating length specifically needs a joint model.
- **Calibration is poor across the board** (ECE 0.22–0.45). If the pitch is reusing DPO checkpoints as reward models, near-random calibration undercuts both methods.
- **Compute is data-matched, not FLOP-matched.** DPO runs an extra frozen reference forward and took $\sim 1.7\times$ the wall-clock of the reward model.

10 Next

- Re-run the headline comparison at each method's own optimum (epoch 2 for the reward model, epoch 1 for DPO) rather than at a shared arbitrary budget.
- Confirm the convergence result with length-controlled rather than raw accuracy, since the reward model's climb could partly be sharpening its length preference.
- Replace length filtering with length adjustment to recover power on the small sets.
- Reseed π_{ref} so the intervals describe the method rather than one reference.
- Then the planned ablations: $\beta \in \{0.05, 0.1, 0.3, 0.5\}$ and budget $\in \{2k, 8k, 32k\}$.

Reproducibility

Code and configs: github.com/silvererudite/dpo-implicit-reward | checkpoints, per-seed results and figures: huggingface.co/Shamima/dpo-implicit-reward | training curves: W&B project *dpo-implicit-reward*. Every checkpoint carries a *budget.json* recording steps, epochs, effective batch, LR, seed and wall-clock, and per-pair scores are saved so every metric here is recomputable without re-running models. Figures regenerate with *reports/make_figures.py*; this document with *reports/make_team_report.py*.